

Banco de dados (Supabase / PostgreSQL)

A estrutura inteira do banco é definida por migrations versionadas em [supabase/migrations/](#), na ordem dos nomes de arquivo (timestamp no início). O arquivo [supabase/scripts/00_estrutura_completa_projeto_novo.sql](#) concatena todas elas, na ordem certa, pra criar a estrutura completa de uma vez num projeto Supabase novo (cole no SQL Editor e rode).

Regra do projeto: **toda regra de negócio e de segurança fica no banco**, não no frontend. RLS, triggers e RPCs `security definer` são a fonte da verdade — a UI só reflete essas regras pra dar feedback melhor ao usuário.

Para o diagrama entidade-relacionamento e a lista completa de chaves estrangeiras (quem referencia quem, obrigatório ou opcional, o que acontece em cada `ON DELETE`), veja [mapeamento-banco-de-dados.md](#).

Tabelas

Referência (cadastros simples)

Tabela	Colunas principais	Observação
<code>departments</code>	<code>name</code> (citext, único), <code>active</code>	
<code>locations</code>	<code>name</code> (citext, único), <code>code</code> , <code>active</code>	
<code>asset_categories</code>	<code>name</code> (citext, único), <code>active</code>	Categorias de acessório (Mouse, Teclado...)
<code>carriers</code>	<code>name</code> (citext, único), <code>active</code>	Operadoras de telefonia

`citext` é usado no `name` pra impedir duplicidade só por diferença de maiúscula/minúscula (ex.: "São Paulo" vs "são Paulo"). Nenhuma dessas tabelas tem exclusão física — só `active = false` (soft delete).

employees — colaboradores responsáveis por ativos

Importante: `employees` é uma entidade **diferente** de `app_profiles` (usuários que logam no sistema). Um colaborador pode nunca ter acesso ao app; um usuário do app pode não ser responsável por nenhum ativo. As duas coisas são propositalmente separadas.

Colunas: `name`, `username` / `email` (citext, únicos, opcionais), `department_id`, `location_id`, `is_shared_asset_holder` (true pra responsáveis não-humanos, tipo "Estoque TI" ou uma sala de reunião — existiam na planilha original), `active`.

Ativos: `notebooks`, `accessories`, `phone_lines`

As três seguem a mesma matriz de campos/regras:

- `status` (enum próprio por tabela — ver abaixo), `employee_id` (ou `assigned_employee_id` em `phone_lines`), `location_id`, `notes`.
- **Soft delete** via `deleted_at` (nunca DELETE físico pela API).
- Identificadores únicos (`patrimonio` / `serial_number` / `number`) são **UNIQUE** de verdade (não parcial) — decisão tomada na migration `20260811150000` especificamente pra funcionar com `upsert` do PostgREST/supabase-js, que só reconhece constraints de unicidade sem predicado.

Tabela	Enum de status	Campos específicos
<code>notebooks</code>	<code>ASSIGNED</code> , <code>AVAILABLE</code> , <code>BROKEN</code> , <code>MAINTENANCE</code> , <code>RESERVE</code> , <code>DECOMMISSIONED</code>	<code>patrimonio</code> , <code>serial_number</code> , <code>modelo</code> , <code>categoria</code> , <code>data_aquisicao</code> , <code>garantia_fim</code>
<code>accessories</code>	<code>ASSIGNED</code> , <code>AVAILABLE</code> , <code>BROKEN</code> , <code>DECOMMISSIONED</code> , <code>MAINTENANCE</code>	<code>category_id</code> (FK), <code>patrimonio</code> / <code>serial_number</code> nullable (maioria dos acessórios não tem identificador físico)
<code>phone_lines</code>	<code>ASSIGNED</code> , <code>AVAILABLE</code> , <code>SUSPENDED</code> , <code>CANCELLED</code>	<code>number</code> (valida formato E.164 via <code>CHECK</code> : <code>^\+[1-9][0-9]{7,14}\$</code>), <code>carrier_id</code> , <code>chip_type</code> , <code>iccid</code> , <code>imei</code> , <code>eid</code>

Ciclo de vida do notebook: não é uma coluna gravada — é calculado on-the-fly pela função `notebook_lifecycle(data_aquisicao)` (vida útil = 4 anos: `TROCAR` se já venceu, `ATENCAO` se vence nos próximos 2 meses, senão `DENTRO_DA_VIDA_UTIL`) e exposto pela view `notebooks_with_lifecycle`, que já filtra `deleted_at is null`. Isso evita o problema que existia na planilha original, onde a coluna de status de garantia ficava incoerente com a data real.

phone_invoices e invoice_uploads — faturas de telefonia

- **phone_invoices**: **phone_line_id** (nulo quando o número faturado não bate com nenhuma linha cadastrada — dado real sem contrapartida, fica pra revisão), **raw_number**, **carrier_id**, **amount**, **invoice_date**, **upload_id**. Chave de dedupe: **unique(raw_number, carrier_id, invoice_date)**.
- **invoice_uploads**: rastreia cada upload de PDF processado pela IA — **status** (**PROCESSING** / **DONE** / **ERROR**), **lines_extracted**, **lines_matched**, **error_message**. Só é gravada pela Edge Function **process-invoice** (service role) — não existe policy de insert/update pro cliente.

app_profiles — usuários da aplicação

```
id          uuid PK, references auth.users(id) on delete cascade
name, email
department_id, location_id
role        USER | ADMIN | MASTER
status      PENDING | APPROVED | REJECTED | BLOCKED
reviewed_by references app_profiles(id) on delete set null
reviewed_at
```

Criada automaticamente pelo trigger **handle_new_user** (**after insert on auth.users**) — ver [#bootstrap-do-usuário-master](#).

audit_logs e asset_movements — as duas trilhas de histórico

Propositamente **duas tabelas diferentes**, com propósitos diferentes:

- **audit_logs**: auditoria técnica automática — **entity_type**, **entity_id**, **action** (**CREATE** / **UPDATE** / **ASSIGN** / **UNASSIGN** / **DEACTIVATE** / **REACTIVATE** /ações de usuário como **APPROVE_USER**, **CHANGE_ROLE** ...), **old_value** / **new_value** em JSONB (diff completo). Gravada só por triggers/RPCs **security_definer**, nunca pelo cliente diretamente. Só MASTER pode ler (**audit_logs_select_master**).
- **asset_movements**: histórico operacional de **atribuição** de um ativo (**asset_type**, **asset_id**, **from_employee_id**, **to_employee_id**, **movement_type**), mostrado na tela do próprio ativo. Qualquer usuário aprovado lê.

Triggers genéricos (**audit_asset_trigger** para notebooks/accessories/phone_lines, **audit_employee_trigger** para employees) decidem a **action** sozinhos comparando **OLD** / **NEW** (ex.: **employee_id** que era nulo e passou a ter valor = **ASSIGN**).

Enums

`notebook_status`, `notebook_lifecycle_status`, `accessory_status`, `phone_line_status`, `app_role`, `profile_status`, `invoice_upload_status` — todos `CREATE TYPE ... AS ENUM`, definidos junto com a tabela que os usa.

RLS (Row Level Security) — matriz geral

RLS está habilitada em toda tabela de domínio. O padrão que se repete:

- **SELECT**: liberado pra qualquer usuário com `is_approved()` (função helper que checa `app_profiles.status = 'APPROVED'` do usuário autenticado).
- **INSERT/UPDATE**: exige `is_admin_or_master()` (aprovado E role `ADMIN` ou `MASTER`).
- **DELETE**: **não existe policy de delete** em nenhuma tabela de domínio — exclusão física é impossível via API; tudo é soft delete (`active` / `deleted_at`).

Exceções à regra geral:

Tabela	Diferença
<code>app_profiles</code>	Sem policy de insert/update pra ninguém — só as RPCs abaixo escrevem. Select: o próprio usuário vê seu perfil; MASTER vê todos.
<code>audit_logs</code>	Select só pra MASTER. Insert só via trigger (<code>security definer</code>).
<code>asset_movements</code>	Sem policy de insert pro cliente — só via as RPCs <code>assign_*</code> / <code>unassign_*</code> .
<code>invoice_uploads</code>	Select só pra MASTER. Sem insert/update pro cliente — só a Edge Function (service role).
<code>phone-invoices</code> (Storage bucket)	Insert/select só pra MASTER.

As funções helper (`current_user_role()`, `current_user_status()`, `is_approved()`, `is_admin_or_master()`, `is_master()`) ficam centralizadas em uma migration só (`20260811140050`) — usadas por praticamente toda policy do projeto, pra não repetir a mesma subquery em cada uma. São `security definer` com `search_path` fixo, pra não depender da própria RLS de `app_profiles` (o que causaria recursão) nem sofrer sequestro de `search_path`.

RPCs (funções chamadas pelo frontend via

`supabase.rpc( ... )`)

Todas `security definer` — rodam com mais privilégio que o usuário comum, então cada uma **revalida a permissão manualmente** internamente (não confiam na RLS da tabela alvo).

Atribuição de ativos

`assign_notebook`, `unassign_notebook`, `assign_accessory`, `unassign_accessory`, `assign_phone_line`, `unassign_phone_line` — cada uma faz, numa transação só: (1) exige `is_admin_or_master()`; (2) atualiza o ativo (`employee_id` + `status`); (3) grava uma linha em `asset_movements`. Ficam numa função (não em dois passos separados no frontend) justamente pra garantir que as duas escritas aconteçam juntas ou nenhuma aconteça.

`deactivate_employee(id)` — desativa o colaborador e libera (`status = 'AVAILABLE'`) todos os notebooks/acessórios dele, mas **não zera** o `employee_id`: o último responsável fica registrado pra consulta histórica (decisão explícita de produto).

Administração de usuários

`approve_user`, `reject_user`, `block_user`, `change_user_role` — todas exigem `is_master()`. `change_user_role` tem duas guardas extras: não deixa alguém alterar a própria role, e não deixa remover o último MASTER `APPROVED` do sistema (evitaria lock-out completo). Toda chamada grava em `audit_logs`.

Criar usuário, resetar senha e excluir conta **não são RPCs** — são feitas pela Edge Function `admin-users`, porque mexem em `auth.users` e exigem a Service Role Key (ver [arquitetura.md](#) e [credenciais-e-seguranca.md](#)).

Bootstrap do usuário MASTER

O trigger `handle_new_user` (dispara em `after insert on auth.users`) cria a linha em `app_profiles` pra todo novo usuário. Ele tem um caso especial: se o e-mail cadastrado bater (case-insensitive) com o e-mail configurado como "MASTER inicial" dentro da própria função, o perfil já nasce `role = 'MASTER'` e `status = 'APPROVED'` — sem precisar de ninguém aprovar. Qualquer usuário criado depois disso segue o fluxo normal (`PENDING` / `USER`), e trocar quem é MASTER a partir daí passa exclusivamente pela RPC `change_user_role`.

Esse e-mail está **hardcoded na função**, não em uma tabela nem variável de ambiente — pra mudar, é preciso alterar e reaplicar a função `handle_new_user()` (está na migration `20260811140000_app_profiles.sql` e no bundle `00_estrutura_completa_projeto_novo.sql`). Veja [credenciais-e-seguranca.md](#) pra saber qual e-mail está configurado hoje e como trocá-lo com segurança.

Storage

Um bucket privado: `phone-invoices` — guarda os PDFs de fatura enviados pra leitura por IA. Só usuários MASTER podem enviar (`insert`) ou ler (`select`) objetos desse bucket (policies em `storage.objects` filtrando `bucket_id = 'phone-invoices' and public.is_master()`).

Views

`notebooks_with_lifecycle` — ver seção de notebooks acima. Criada com `security_invoker = true` de propósito, pra respeitar a RLS da tabela base (notebooks) em vez de herdar os privilégios de quem criou a view.

Extensões usadas

- `pgcrypto` — `gen_random_uuid()` (chave primária de toda tabela) e `crypt()` / `gen_salt()` (usado só em scripts de seed manuais pra criar um usuário direto via SQL, nunca em runtime).
- `citext` — comparação case-insensitive nativa em nomes/e-mails/usernames.